ECE 49595 Term Project: Urban Tree Classifier

Arushi Gupta, Javier Perez Calderon, Ashna Sinha

chanical Engineering

Introduction

Objective

- We built a 23-class urban tree image classification model.
- Each class corresponds to a different tree species category.

Dataset

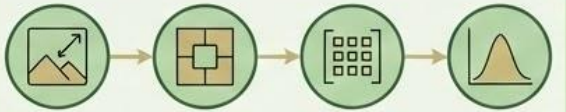
- Given RGB images were sorted into 23 plant species with each image pertaining to one plant species.

Approach

- We used a transfer learning approach by fine-tuning a pretrained ResNet-18 model on our dataset.
- Input is a 3 × 224 × 224 image and output is a 23 class logit

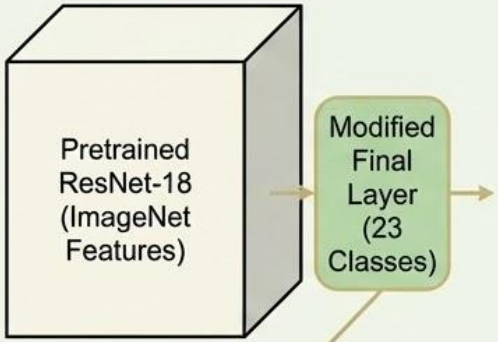
Methodology

Data Preparation and Processing



- **Resize + CenterCrop (224×224):** ResNet requires fixed-size inputs.
- **ToTensor:** Converts images to PyTorch tensors.
- **Normalize(mean, std):** Matches ImageNet distribution for stability.

Model Architecture




```
num_features = model.fc.in_features
model.fc = nn.Linear(num_features, 23)
```

Replaced final fully-connected layer for 23-class output.

Loss Function & Optimization

Loss:
CrossEntropyLoss (Standard for multi-class classification)

Optimizer:
Adam (learning rate = 0.001) (Faster convergence than SGD)

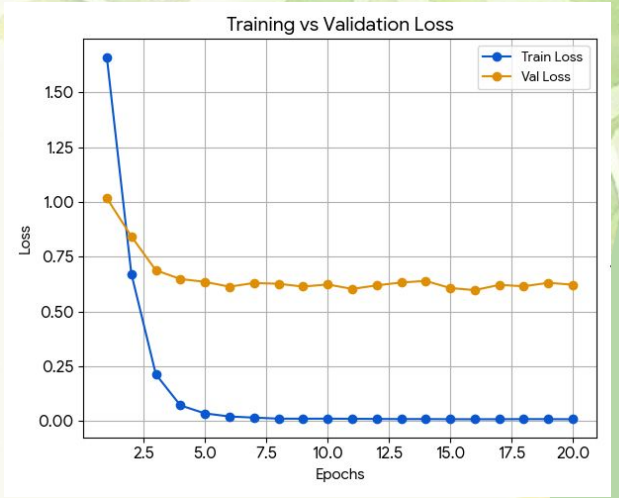
Batch size: 32

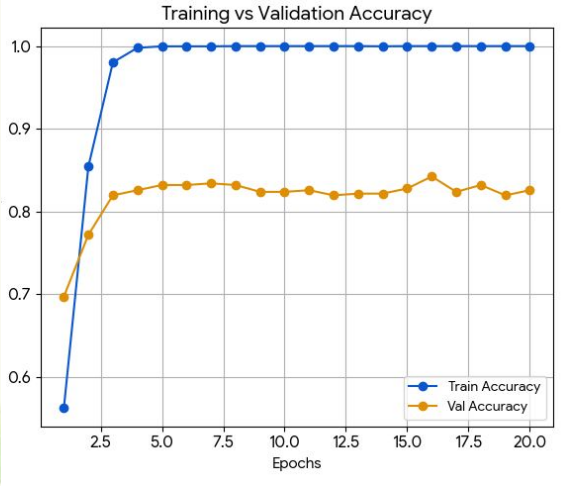
Epochs: 20

Training Procedure

Main steps performed each epoch:

1. Set model to **train mode**
2. Load minibatches from training DataLoader
3. Forward pass:
   ```
   outputs = model(images)
   ```
4. Compute loss:
   ```
   loss = criterion(outputs, labels)
   ```
5. Backpropagation:
   ```
   loss.backward()
   ```
6. Update weights:
   ```
   optimizer.step()
   ```
7. Track training loss





Validation & Testing

Validation Loop

After each epoch:

- Model set to **eval mode**
- No gradient computation
- Compute accuracy on validation set
- Used to monitor overfitting

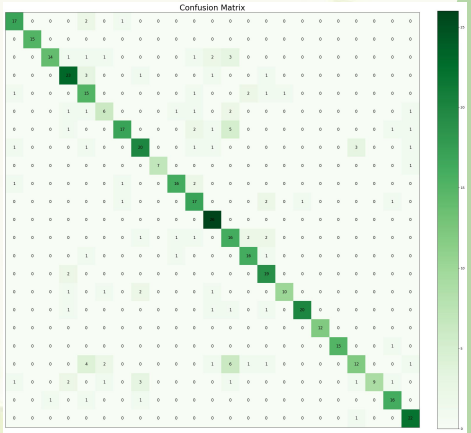
Why have validation?

- Helps tune learning rate
- Helps detect overfitting
- Allows model selection if multiple checkpoints are saved

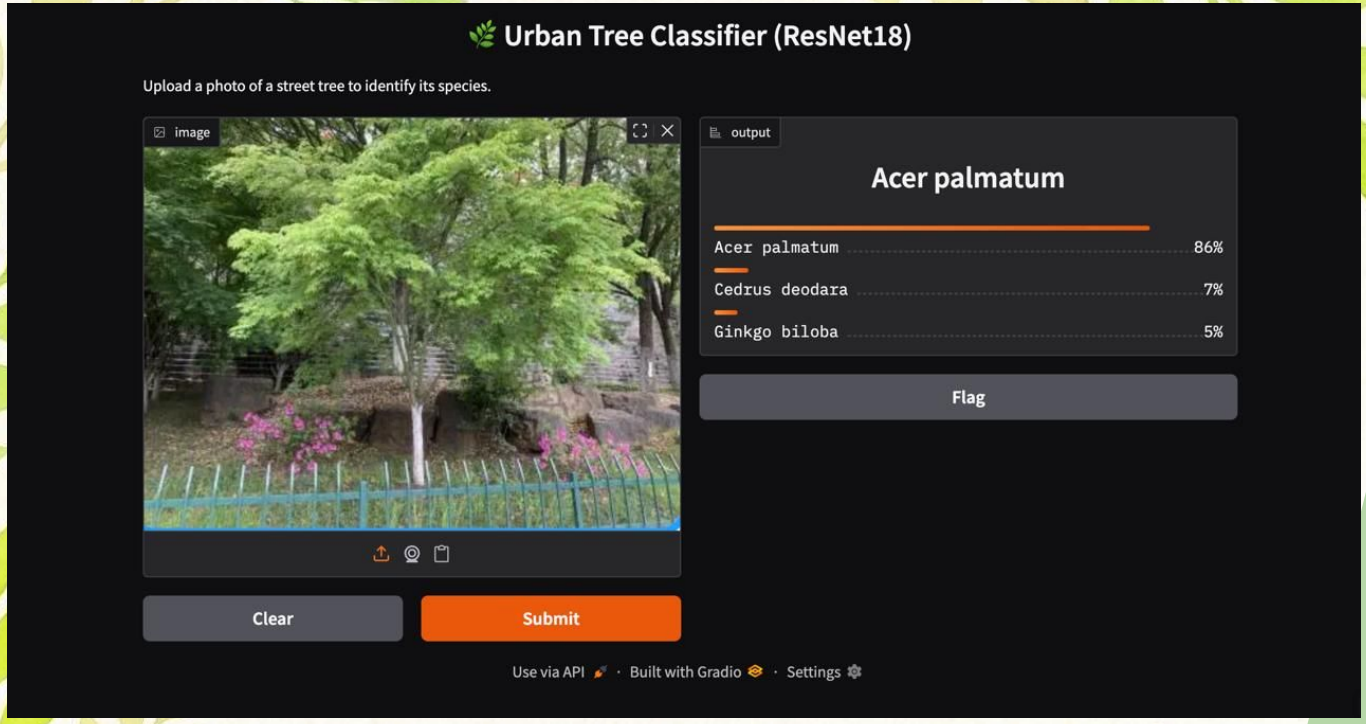
Testing Procedure

Final evaluation on unseen data:

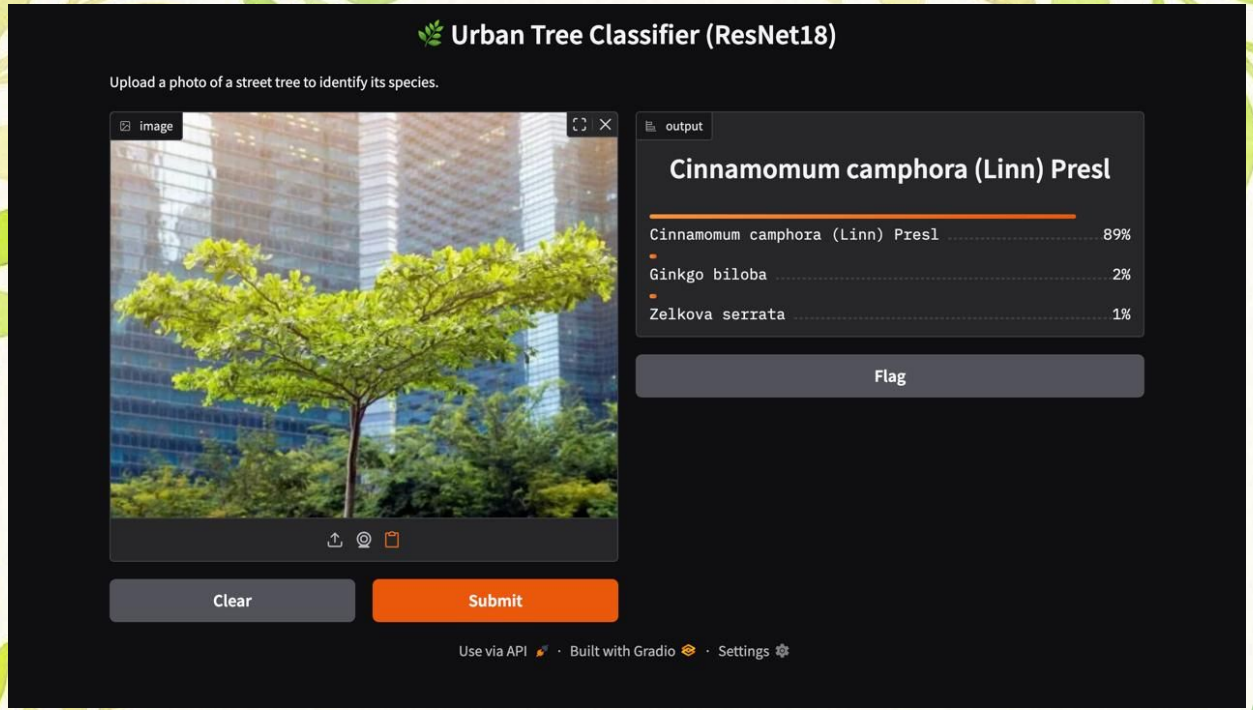
- Similar to validation loop
- Computes final test accuracy
- Predictions collected for confusion matrix
- Measures generalization ability



Examples


Urban Tree Classifier (ResNet18)
Upload a photo of a street tree to identify its species.
image
output
Acer palmatum
Acer palmatum 86%
Cedrus deodara 7%
Ginkgo biloba 5%
Flag
Clear
Submit
Use via API · Built with Gradio · Settings

Examples


Urban Tree Classifier (ResNet18)
Upload a photo of a street tree to identify its species.
image
output
Cinnamomum camphora (Linn) Presl
Cinnamomum camphora (Linn) Presl 89%
Ginkgo biloba 2%
Zelkova serrata 1%
Flag
Clear
Submit
Use via API · Built with Gradio · Settings

Technical Strengths & Improvements

- Transfer learning dramatically reduces training time
- Pretrained ResNet backbone extracts strong features
- Clean separation of train/val/test sets
- Consistent preprocessing → stable model behavior
- Confusion matrix gives deep interpretability

1. Use RandomHorizontalFlip, RandomRotation for augmentation
2. Add EarlyStopping
3. Add lr_scheduler (StepLR or ReduceLROnPlateau)
4. Try deeper models (ResNet-34, DenseNet-121)
5. Fine-tune more layers instead of only the final layer
6. Balance dataset with oversampling or class weighting

Thank You

PURDUE UNIVERSITY | School of Mechanical Engineering